

LumoTree: Path-Parallel Speculative Verification for Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

Abstract—Tree speculative decoding for hybrid language models must preserve one coherent continuation across recurrent, convolution, and attention state. We present LumoTree, a verifier that executes recurrent paths in parallel and coordinates native recurrent replay, convolution-history gathering, and attention-cache remapping through a shared logical tree. Fused candidate selection, GPU-resident acceptance, and grouped split-K attention support the verification cycle. On Qwen3.8-27B NVFP4, 126 continuation cycles satisfy frozen numerical bounds relative to native controls, with all ten injected faults detected. On a single NVIDIA DGX Spark (GB10), two disjoint 43-request corpora record 28.79 versus 26.12 and 30.74 versus 27.52 pooled tokens/s against five-token native multi-token prediction, with three runs per arm. A separate 30-attempt coding-agent study on ten SWE-bench Verified tasks does not reproduce this decode-rate ranking. Exact sampling is claimed only for penalty-free settings with matching probability filters, conditional on correct target probabilities. The deployed and coding-agent runs use presence penalty 1.0; incorrect tree histories make their sampling approximate after the first token of each step. These results characterize the implemented system’s replay rates and bounded numerical agreement, without establishing lossless deployed sampling or task-completion speedup.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

LumoTree executes recurrent paths in parallel, reuses state tiles across updates within each path, and coordinates the accepted continuation across recurrent, convolution, and attention state. An ancestry-aware attention path and a device acceptance walk use the same logical tree as recurrent verification and state publication. A shared descriptor carries ancestry and logical-to-physical mappings through each stage. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. The verifier scans branch paths, while the fixed-shape committer replays accepted operands through native GDN updates in a captured graph. Their finite-precision agreement

is a separate requirement, not assumed from the recurrence. This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry, how to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper studies three coupled design questions in recurrent-hybrid tree verification. *First*, how can branch-local computation produce one coherent continuation across recurrent, convolution, and attention state? We define a shared logical tree and coordinate state publication through its logical-to-physical mapping. *Second*, how should verification trade temporary state storage against repeated computation? We use path-parallel scans with shared boundary states and replay the accepted path into persistent state, building on established chain-scheduling and replay mechanisms [6], [7]. *Third*, how can this organization reduce serving overhead? We combine fused candidate selection, device-resident acceptance, and grouped split-K attention with a spine-contiguous KV layout. We evaluate component numerical agreement, recorded-request replay against native decoding, and an exploratory coding-agent deployment. Section IX bounds these results.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [8]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Candidate-selection and attention tests complement the agent-workload measurements. Sampling exactness is restricted to penalty-free settings with matching probability filters and correct target probabilities. The measured presence-penalty-1.0 deployment departs from exact sampling after the first token of each tree step (Section VII-F).

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms. Sections III–IV develop the architecture and kernel design; Section V describes the optimizations that connect them to the serving cycle. Appendix A gives configuration and measurement details.

II. BACKGROUND AND RELATED WORK



Fig. 1. Tree verification and accepted-path commitment. (a) Drafting provides a candidate tree and proposal probabilities q . (b) Branch-local computation produces target probabilities p and retains temporary state. (c) Sampling selects a materialized path (here a, d); recurrent replay, convolution gathering, and KV remapping publish $S(mad)$. The newly sampled token b^* remains pending. One descriptor keeps logical ancestry and physical storage aligned.

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [9]–[11]. EAGLE-3 and DFlash further change the drafter, through feature-conditioned autoregression and block-parallel diffusion proposals respectively [12], [13]. These drafting mechanisms are distinct from the recurrent verifier and state-publication choices studied here.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [14], [15]. Marconi addresses prefix-cache admission and eviction for hybrid models [16]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [17], [18]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. Flash-Decoding splits the KV sequence and combines partial attention results; FlashInfer develops customizable attention with head/query fusion and split-KV execution; DeFT specializes KV grouping and partitioning to trees [19]–[21]. FastTree groups queries with shared contexts to reuse KV tiles and partitions tree work to control execution cost [22]. These attention methods establish precedents for on-chip reuse and tree-aware partitioning. Our modified FlashAttention-2 kernel combines ancestry-aware decode, grouped split-K execution, and a physical layout coordinated with accepted-path publication [23].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [24]. Snakes and Ladders introduces activation replay for SSM speculative decoding, and STree also uses replay to recover continuation state [5], [25]. Stateful speculation is established territory:

STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [26], [27]. These works motivate an explicit state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Trees from Marginals (Weaver) combines a factorized drafter with an autoregressive adapter. Its GDN verifier uses an ancestor-masked triangular solve and replays the selected path through a short recurrence at commit time; padded verification shapes support CUDA graphs [7]. Thus read-only speculative verification followed by accepted-path replay is direct prior art, even when the verification arithmetic differs from ours.

SpecLA keeps value tiles of recurrent state on chip across updates within a chain and schedules independent ready chains in parallel. Boundary states connect dependent chains. This directly overlaps both the state reuse and path scheduling in LumoTree. SpecLA buffers accepted factors for a later fused update, whereas our publication stage replays recorded operands through native GDN updates into running rows. Its evaluation uses a pure GDN model; our serving boundary additionally coordinates convolution and softmax attention [6]. SpecLA’s token-replay baseline regenerates factors, so its cost cannot be assigned to our replay of cached operands.

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It reuses intermediate tiles on chip, stores compact update factors, and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-72B on GB10 and replay of recorded OpenHands agent sessions [28]. This is a direct hybrid-serving comparison. Our evaluation reports recorded-request replay separately from running the coding agent on tasks.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U =$

TABLE I
RECURRENT VERIFICATION AND CONTINUATION POLICIES. THESE
MECHANISM COMPARISONS DO NOT RANK THE SYSTEMS BY
PERFORMANCE.

Method	Verification work	Continuation state
Weaver [7]	Ancestor-masked triangular solve	Short selected-path recurrence
SpecLA [6]	Resident state tiles; parallel chains	Buffered factors; later fused update
Bole [28]	Finite-Neumann correction solve	Compact factors; state reconstruction
TreeWY [30]	Triangular solve in fused kernel	Prior-path reconstruction with next verify
ReplaySSM [31]	Corrected inputs; cached transitions	Deferred state update
OneLA [32]	Shared-base projections across beams	Compact per-beam transition records
LumoTree	Resident state tiles; parallel paths	Native replay; coordinated convolution and KV publication

$\sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [28]. Our path preserves sequential updates and replays accepted operands; Section IV-B explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. The author implementation fuses reconstruction of the previous accepted state with the next verification, reusing a resident state tile. Its GDN tree kernel supports CUDA graph capture; the paper’s wider-tree fallback to piecewise execution concerns ancestry masks in the softmax-attention integration [29], [30]. Its B200 evaluation uses greedy verification and disabled prefix caching. A comparison must therefore include fused commit work and distinguish kernel support from whole-model execution policy.

ReplaySSM caches recurrent inputs and delays state materialization. Its GDN path uses corrected inputs from a triangular solve, with commitment represented through buffer progress and later state updates [31]. **OneLA** addresses a related state-sharing problem in recommendation beam search: compact transition records and ancestry indices allow projection computation without reconstructing a full state for each beam, and a fused kernel shares the prompt-derived state across beams [32]. Its beam-search output and commitment policy differ from speculative acceptance. Table I compares state lifetimes; published speed ratios from different models and workloads are not combined into a ranking.

D. Design focus

LumoTree connects path-parallel recurrent computation to one accepted continuation across the hybrid model. The shared descriptor links path boundaries, convolution ancestry, attention visibility, physical KV slots, and the next drafter input. Native replay preserves the serving system’s recurrent-state ownership while selected-path gathering and remapping publish the other state components. Spine-first KV placement

controls the physical attention reduction layout within this design, building on established floating-point reduction and batch-invariance concerns [33]. Evaluation separates component behavior from complete agent-workload performance.

III. TREE-VERIFIER ARCHITECTURE

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$\mathcal{S}(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(\mathcal{S}(m), b)$, not a claim that $\mathcal{S}(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces coordinated by the verifier.

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$\begin{aligned} R'_i &= \alpha_i R_{\pi(i)}, & \delta_i &= \beta_i (v_i - R'_i k_i), \\ R_i &= R'_i + \delta_i k_i^\top, & o_i &= R_i q_i, \end{aligned} \quad (2)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. A descriptor shared by verification and commitment

Figure 1 shows how a shared descriptor connects drafting to continuation. Native multi-token prediction (MTP) supplies a principal chain and runner-up candidates; suffix prediction extends selected paths within a bounded candidate capacity [34],



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

[35]. Combining retrieval and neural drafting has prior art, including SAM Decoding [36]. The verifier derives attention visibility, physical packing, path scheduling, and the acceptance walk from the same logical topology. Padding permits reusable execution shapes while remaining invisible to attention and selection. The particular tree geometry used in our experiments is specified in Section VI.

The attention layout places the principal spine first in the physical suffix and permutes KV write locations and ancestry-mask key columns while retaining logical query-row and position order. Selection remains in logical-node space; accepted-path publication maps those nodes back to their physical slots. This distinction prevents a layout optimization from silently changing which key/value entries a branch reads. Recurrent paths begin at the native pre-tree state or a transient cut state; the device committer publishes the selected recurrent and convolution state into native running rows and re-linearizes its attention KV entries.

Algorithm 1 states the intended probability and continuation contract and separates candidate verification, path selection, and state publication. The deployed history-dependent logits processing does not satisfy its intended target-probability assumption beyond the first token of each step; Section VII-F audits that defect. Every publication operation uses the same materialized path A . A newly sampled token b^* can be emitted while remaining pending; its state is created by the next target forward.

IV. GPU SCAN AND ACCEPTED-PATH REPLAY

A. Path-parallel verification and state lifetime

The GDN verifier decomposes the tree into dependent path groups, a scheduling family also studied by SpecLA [6]. A first path starts from the request’s pre-tree state and exports the cut states needed by downstream paths. The second level evaluates independent branch paths from those cut states. Each GPU program covers one path, head, and value tile, retaining the full key dimension in its working state tile. Sequential updates reuse this tile through the path loop rather than loading a full recurrent state for every node. Candidate outputs continue through the model; rounded operands are recorded for later accepted-path replay. The source expresses this local reuse; register allocation and spills remain compiler- and geometry-dependent.

Cut states and operand rings are temporary within a verification step. The durable continuation resides in the request-owned native recurrent and convolution running rows. Replay starts from the pre-tree running state and publishes the accepted frontier back to those rows; prefix-cache snapshots and restores use that native layout. Static work buffers can be reused by CUDA graphs, but they do not replace the request’s persistent state with a branch-leaf pointer.

The two levels execute in separate GPU launches and exchange only the boundary states required by downstream paths. This schedule exposes parallel branch work while retaining the state-transfer cost at the decomposition boundary.

B. Sequential arithmetic and accepted-path replay

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. Accepted-path replay uses native fused GDN updates with recorded accepted operands and in-place publication to the fp32 state bank. Its graph contains one native update call per GDN layer. Query/key normalization, beta conversion, gate evaluation, and output casts can differ between these implementations; agreement is not guaranteed by their common real-arithmetic recurrence.

The sampling stage produces accepted node indices and path lengths on the GPU. Recurrent replay, convolution publication, and attention-KV publication consume this same path. Graph capture reduces host launch overhead for the sequence of per-layer native updates; each layer retains its own update kernel.

C. Temporary state versus recomputation

Path decomposition caches the states needed where downstream paths branch, then reuses them across independent work. If a program exports N_c cut states with value-tile width B_v and key dimension d_k , their fp32 payload is

$$M_{\text{cut}} = 4N_c B_v d_k \text{ bytes.} \quad (4)$$

This counts one head and value tile’s logical export payload. The current reusable scratch allocation reserves a full matrix slot for each actual node even though only required cut states are written; allocated bytes therefore differ from state traffic. Recomputing ancestry could remove some exports at the cost of repeating updates. Exporting every candidate state would instead trade more writes for less recomputation. The two-level path schedule occupies a particular point in this design space.

Accepted-path replay carries one state tile through a bounded path loop. Its carried-state footprint is $O(B_v d_k)$ in tree size, while the recorded operands and candidate outputs scale with the verification shape. Static padding makes graph buffers and kernel geometry reusable, but padded rows must remain invisible to attention and selection. Neither a shorter logical path nor fewer state writes alone establishes a faster complete request.

The closest scheduling comparison is SpecLA: both systems carry state tiles through sequential paths and parallelize ready paths. Their continuation policies differ. SpecLA buffers accepted factors for application in a later fused update [6];

Algorithm 1 Tree verification and native continuation

Input: Native continuation state; candidate tree T ; proposal probabilities q

Output: Emitted tokens; updated native state; next drafter input; any pending token

Verify candidates

- 1: Map T to physical slots, positions, and ancestor masks.
- 2: $p \leftarrow$ target probabilities from path-local GDN, convolution, and attention computation.
- 3: Retain candidate hidden states, rounded replay operands, convolution histories, and temporary KV entries.

Select one continuation

- 4: $(A, b^*) \leftarrow$ accepted materialized path and pending token from sampling with p and q .

Publish the same path across all state components

- 5: **for** each GDN layer **do**
 - 6: Replay recorded operands along A into the native recurrent state.
 - 7: Gather the convolution history for A into the native running row.
 - 8: **end for**
 - 9: Remap the accepted KV entries using the same logical-to-physical slot map.
 - 10: Select the matching hidden-state row for the next drafter input.
 - 11: Advance positions along A ; keep b^* pending until its target forward.
 - 12: **return** Emitted tokens and the updated continuation boundary.
-

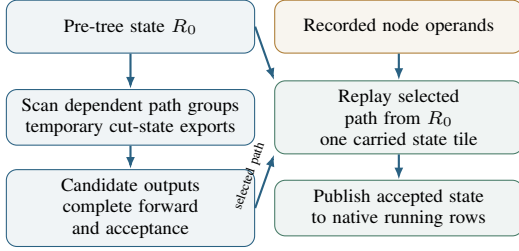


Fig. 3. State lifetime in the path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

LumoTree immediately replays recorded operands through native per-layer updates and publishes the matching convolution and attention state. Weaver also uses selected-path replay, but verifies candidates with a triangular solve [7]. Bole reconstructs accepted states from compact factors in a batched commit [28]. TreeWY can fuse prior-frontier reconstruction with the next verification while retaining its working tile [30]. Thus, LumoTree pairs path recurrence with immediate native replay and coordinated hybrid-state publication. This pays explicit accepted-path update work in exchange for returning each iteration to the native state representation. Complete-cycle measurements must include that tradeoff.

V. OPTIMIZING THE LUMOTREE VERIFICATION CYCLE

The architectural choice in Section IV is realized through the implementation changes below. Three representations must agree: logical tree nodes, temporary verification storage, and the native state used by the next forward. LumoTree carries one accepted path across these representations and reorganizes preparation and selection around it, using established techniques such as fusion and graph replay. The nontrivial constraints arise at their boundaries: changing a slot order, a rounding

step, or a tied candidate can change which continuation the next iteration consumes. Table II summarizes the changes and these constraints.

A. One device-selected path drives native continuation

Selecting valid output tokens does not by itself update a hybrid model’s state. The recurrent matrix must contain the accepted updates, convolution must retain the accepted history, attention must reference the accepted KV entries, and the drafter must receive the corresponding hidden row. Reconstructing these choices separately on the host introduces path-list preparation and synchronization between acceptance and publication.

LumoTree replaces that handoff with fixed-shape device products: accepted node indices, path lengths, emitted tokens, and the selected final row. These products feed native recurrent replay, convolution gathering, KV remapping, and drafter-row selection. Recorded operands are staged for the native per-layer updates in one captured committer graph; convolution and KV publication use the same selected path. Graph capture amortizes launch orchestration, while the shared path prevents each state component from independently interpreting acceptance.

The distinction between emitted and materialized tokens must survive this handoff. A correction or bonus token may be emitted while remaining pending for the next target forward; padded path entries must not advance state. After publication, the implementation stores persistent state in native request rows and branch cut states in temporary scratch. The native representation supports the intended prefix-cache and serving-graph interface. The implementation captures the serving, drafter, and committer stages separately.

B. One slot map governs attention and publication

An ancestry mask specifies visible keys, but does not specify their physical placement or the reduction groups that process

Algorithm 2 Path-parallel scan and accepted-path replay

Input: Candidate tree T and the pre-tree recurrent state for each GDN layer

Output: Recurrent outputs for all candidates; native recurrent state for the accepted path

Verify paths; keep branch state temporary

```
1: for each GDN layer reached during the target forward do
2:   Compute and record this layer's rounded node operands.
3:   for each path group, in dependency order do
4:     parallel for each ready path, head, and value tile do
5:       Load the working state tile from the pre-tree state or a saved parent cut state.
6:       for each node on this path, in ancestry order do
7:         Update the working tile with this node's operands; write its recurrent output.
8:         Save the tile if a downstream path needs this node's state.
9:       end for
10:    end for
11:  end for
12: end for
```

Finish verification and select a path

13: Complete the target forward and sampling to obtain the accepted materialized path A .

Replay only the accepted path into native recurrent state

```
14: for each GDN layer do
15:   Start from this layer's pre-tree recurrent state.
16:   for each node on  $A$ , in path order do
17:     Apply the native GDN update using this layer's recorded operands.
18:   end for
19:   Publish the resulting recurrent state to the request's native running row.
20: end for
```

TABLE II

CONCRETE IMPLEMENTATION CHANGES AROUND THE PATH VERIFIER. THE THIRD COLUMN IDENTIFIES THE COORDINATION OR NUMERICAL CONSTRAINT THAT MAKES EACH CHANGE NONTRIVIAL. WORKLOAD RATES MEASURE THEIR COMBINED EXECUTION.

Starting work or constraint	LumoTree change	What must remain consistent
Host path-list construction between acceptance and publication	Device-selected path feeds captured native replay, convolution, KV and drafter selection	All consumers select the same materialized prefix; pending and padded tokens do not advance state
Logical ancestry does not determine physical KV placement	Spine-contiguous slots with coordinated writes, mask columns and accepted-path remap	Logical query order, physical source slots and native destination positions agree
Per-node convolution histories and repeated layer preparation	Static ancestry gathers, fused ordered taps and per-layer operand buffers	Tap order, cast boundaries, history padding and layer identity
Separate score preparation and argmax/top-k operations	One shared logits tensor and fused selection into graph buffers	Both reference tie conventions, candidate order and duplicate behavior
Repeated KV staging and limited decode parallelism	Grouped query heads and split-K in the patched tree-attention kernel	Tree visibility and slot mapping across partitions; characterized reduction error

them. Interleaving masked branch columns with spine keys can change the online-softmax reduction layout even when logical visibility is unchanged. Physical ordering therefore matters both for addressing and for finite-precision execution [33].

LumoTree places the principal spine contiguously in the candidate suffix while retaining logical query-row and position order. It applies the same permutation to KV write slots and ancestry-mask key columns. Acceptance remains in logical-node space; publication translates the selected nodes through that slot map into native continuation positions. Reordering only writes, only masks, or only the final copy would leave another consumer reading a different branch. The implementation

change is this coordinated mapping across verification and publication.

The patched FlashAttention-2 decode kernel combines this layout with grouped query heads that share staged KV tiles and split-K context partitions. KV reuse and partitioned attention are established techniques [21], [22]; here they are applied under the tree's visibility and slot mapping. They target repeated KV staging and insufficient parallel work at batch one. Prefill retains a native-compatible attention path. The current component tests measure repeatability and numerical error of the deployed attention kernel; the isolated effect of spine placement remains an ablation question.

C. Precompute branch histories while preserving arithmetic

The starting tree-convolution implementation constructs candidate histories with per-node operations and repeats path-row preparation across layers. LumoTree derives static gather indices from the topology, gathers windows from the prior convolution history and candidate inputs, and fuses the tap computation. Stacked operand buffers retain each GDN layer’s inputs for subsequent accepted-path replay.

This transformation must preserve more than ancestry. Convolution tap order and cast boundaries affect the values recorded for recurrent updates, and unused history positions must retain the intended padding. A generic reduction can compute the same real-arithmetic sum in a different floating-point order. The fused implementation therefore retains the ordered tap operations and conversion points while replacing repeated history construction with prepared data movement. Each layer still records and replays its own operands.

D. Fuse draft selection without changing proposals

At each MTP head depth, LumoTree computes one full-vocabulary logits tensor and reuses it for the principal token and sibling candidates. A fused CUDA kernel writes the spine token and three ordered candidate indices directly into reusable graph buffers, replacing separate argmax/top-k reductions and their output copies. The principal chain advances through captured MTP iterations; sibling selection reuses its scores, and suffix prediction extends selected paths within the fixed verification capacity.

The subtlety is that argmax and top-k do not have interchangeable tie behavior. The reference argmax chooses the lowest maximal token index, while its top-k operation orders selected ties differently. Simply taking the first top-k result as the spine token can change the proposal tree. Our fused selection preserves both reference outputs, including duplicate proposals where produced by the reference. Exact-output and captured-replay tests check this component. The optimization removes redundant score preparation and selection work while retaining the vocabulary projection.

VI. EVALUATION SETUP

We evaluate Qwen3.8-27B NVFP4 on a single NVIDIA DGX Spark (GB10) using a coding-agent study, recorded-request replay, full-model continuation diagnostics, and isolated recurrent measurements. Qwen Code performs repository edits and tool calls on ten SWE-bench Verified Astropy tasks [8], once per task under each of three serving arms: LumoTree, native five-token MTP, and SGLang EAGLE. The benchmark evaluator tests nonempty patches; empty patches count as unresolved attempts. Separate replays compare serving configurations on two disjoint corpora of 43 requests each. Decode rates pool token counts and decode durations over the same completed requests, excluding time to first output. Agent duration additionally includes tools and repository operations; server startup and evaluation are separate.

All three arms load the same ModelOpt mixed-precision checkpoint. Target MLP weights and the vocabulary projection

use NVFP4, attention projections use FP8, and activations and KV storage use BF16; recurrent state uses FP32. The checkpoint’s one-layer MTP module remains BF16. Native MTP and LumoTree use this module, and SGLang invokes it through its EAGLE implementation without a separately trained draft checkpoint. Native MTP and LumoTree share the vLLM build; SGLang uses a separate serving stack.

The evaluated tree has 27 draft candidates and one root, padded to 32 verification rows. A five-depth MTP head supplies 15 candidates; suffix prediction adds a six-token principal tail and rescue branches of four and two tokens. Maximum draft depth is eleven. Five MTP passes construct the head, including four forwards after its initial prediction; the deeper suffix extensions do not require additional MTP passes. Attention groups query heads in pairs and divides the context into four partitions. The model has 48 GDN layers and sixteen full-attention layers. All main LumoTree workload runs use this geometry and the patched FlashAttention-2 path. Appendix A gives the serving configuration; the accompanying artifact identifies each source and kernel version.

The first replay corpus contains 43 requests recorded from the Astropy tasks 12907 and 13033. Each configuration receives the same saved messages and tool definitions, sequentially with at most one active request. Reported vLLM prompt lengths range from 23,024 to 68,557 tokens. Sampling uses temperature 0.6, top- p 0.95, top- k 20, and presence penalty 1.0, with a 1,024-token response cap and no per-request seed. This protocol measures fixed-request serving behavior; generated tool calls are not executed. At presence penalty 1.0, the deployed tree sampler’s history processing departs from exact sampling after the first token of each step (Section VII-F); these rates therefore do not compare equal output distributions.

Native autoregressive decoding, chain MTP, and LumoTree use the same checkpoint, vLLM image, tokenizer, chat template, and common serving settings, including prefix caching, chunked prefill, a 131,072-token context limit, and one active sequence. The native arms use their native attention invocation; LumoTree uses its specialized tree-attention path. For the cross-stack comparison, we capture the complete input token sequence from vLLM’s tokenizer endpoint and pass those IDs directly to SGLang’s generation endpoint. All 43 sequences match the vLLM prompt lengths and the SGLang run records. SGLang EAGLE uses the same checkpoint, FlashInfer attention, and its own prefill and streaming implementation.

The disjoint corpus selects 43 of 61 captured requests from native-MTP agent executions on tasks 14096, 14309, 14508, and 14995, across five conversations. Selection allocates requests proportionally across conversations and spaces them through capture order. Its manifest is frozen before any replay; neither message bodies nor initial user messages overlap the first corpus. LumoTree and native MTP-5 replay these saved messages in three runs each; SGLang has one run. SGLang uses its chat endpoint for this corpus, so matching messages does not establish matching serialized input IDs. Input collection is distinct from the 30-attempt task-outcome study.

For vLLM and the disjoint corpus’s SGLang chat endpoint,

TABLE III

TEN-TASK CODING-AGENT STUDY, ONE ATTEMPT PER TASK AND ARM. MINUTES SUM AGENT DURATIONS, INCLUDING TOOLS BUT EXCLUDING SERVER STARTUP AND EVALUATION. DECODE RATES USE SERVER-COUNTER DIFFERENCES IN EQ. 5; THE TREE ARM USES APPROXIMATE PENALTY HISTORIES.

Arm	Resolved	Requests	Tokens/s	Minutes
LumoTree	5/10	236	26.19	144.0
Native MTP-5	6/10	224	27.17	99.5
SGLang EAGLE	5/10	210	29.89	103.0

the replay client measures time from the first nonempty streamed content, reasoning, or tool-call delta to stream completion. For the first corpus’s SGLang token-ID endpoint, timing begins at the first streamed increase in the completion-token count. We apply Eq. 5 to these client durations and server-reported completion counts, pooling matched token and time totals across requests and repetitions. The estimate excludes first-output latency and server startup. Subsequent stream delivery and parser behavior remain in the measured duration; the endpoint observables are not identical.

VII. RESULTS

A. SWE-bench Verified coding-agent workload

The coding-agent study completes 30 attempts: ten selected Astropy tasks, once each under LumoTree, native MTP-5, and SGLang EAGLE. All arms use Qwen Code 0.19.4, the same checkpoint, a 24,000-token response cap, and a 9,000-second task budget. The agent inspects repositories, edits files, and executes tools. Table III summarizes all attempts, including four empty patches; Appendix B gives task-level results. The LumoTree arm uses presence penalty 1.0 with the history-processing defect in Section VII-F, so this is a comparison of deployed procedures, not lossless target sampling.

For completed request r , let n_r be the output-token count, ℓ_r the server-recorded end-to-end latency, and f_r the time to first output. We report the pooled decode rate

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)}. \quad (5)$$

Counts and durations cover the same completed requests, including requests that summarize an overlong agent context. The numerator counts output intervals; the denominator excludes first-output latency and between-request agent/tool work. Thus D_{pool} measures token production over accumulated request durations, rather than aggregate wall-clock throughput.

LumoTree produces ten nonempty patches, five of which fail tests. Native MTP-5 produces six resolving patches, two failing-test patches, and two empty patches; SGLang produces five, three, and two respectively. Empty patches count as failures without invoking the test evaluator. These are descriptive outcomes on the selected tasks, not benchmark-wide quality estimates.

The live agent trajectories differ: LumoTree emits 187,137 output tokens, compared with 134,643 for MTP-5 and 158,827

TABLE IV

SAMPLED REPLAY OF 43 RECORDED REQUESTS, CAPPED AT 1,024 OUTPUT TOKENS. RATES POOL MATCHED TOKEN AND TIME TOTALS ACROSS ALL RUNS. RANGES DESCRIBE INDIVIDUAL RUN RATES, NOT CONFIDENCE INTERVALS. TREE PENALTY HISTORIES ARE APPROXIMATE (SECTION VII-F). SGLANG RECEIVES THE SAME INPUT TOKEN IDS BUT USES A DIFFERENT STACK AND STREAMING ENDPOINT.

Configuration	Runs	Tokens/s	Run range
vLLM autoregressive	1	10.54	—
vLLM MTP, 1 token	1	15.37	—
vLLM MTP, 3 tokens	1	22.06	—
vLLM MTP, 5 tokens	3	26.12	26.04–26.23
vLLM MTP, 7 tokens	1	25.72	—
LumoTree	3	28.79	28.38–29.41
SGLang EAGLE, 7/8	3	29.69	28.91–30.90

for SGLang. Their request counts, prompt lengths, and tool calls therefore differ as well. The lower LumoTree decode rate and longer summed agent duration in this study contrast with the fixed-request replay results below. Sharing tasks and agent settings does not isolate a verifier’s effect on task completion.

B. Recorded-request replay

Table IV reports complete configuration runs, each containing all 43 requests. Across three runs per arm, LumoTree records 28.79 pooled tokens/s and five-token native MTP records 26.12, a 10.2% difference on this corpus. Their individual run rates range from 28.38 to 29.41 and from 26.04 to 26.23 tokens/s, respectively. Plain autoregressive decoding records 10.54 tokens/s in one run. The native MTP sweep uses one, three, five, or seven draft tokens; five-token MTP has the highest observed pooled rate among these settings.

For SGLang, the paired setting denotes speculative steps and draft-token capacity; EAGLE top- k is one. Seven steps and eight draft tokens yield 29.69 pooled tokens/s over three runs with identical incoming token IDs. Attention, prefill settings, penalty-history processing, and endpoint timing still differ (Section IX). The native MTP sweep is exploratory and has unequal repetition counts.

Additional runs disable individual LumoTree optimizations (Table V). The stock-attention variant completes all 43 requests in three runs and records 26.82 pooled tokens/s. Disabling fused draft selection or precomputed tap-operand preparation records 29.40 and 29.02 in one run each. Stochastic output lengths and limited replication prevent attributing these differences to the individual transformations. Phase and boot-allocation observations below characterize selected variants, without a complete per-optimization decomposition.

Phase timers add a descriptive view of these variants (Table VI). Stock attention has a 14.6 ms longer target span and a 15.1 ms longer step than the two instrumented deployed repeats. Disabling fused selection increases the drafter span by 0.7 ms; operand-preparation removal has no clear measured separation. Request inputs are shared, but generated trajectories, acceptance lengths, and actual KV capacities differ, so these are not isolated causal effects at identical computation.

TABLE V

EXPLORATORY REPLAY VARIANTS. EACH RUN CONTAINS ALL 43 REQUESTS. THESE MEASUREMENTS DO NOT ESTABLISH ISOLATED OPTIMIZATION EFFECTS.

Disabled optimization	Runs	Tokens/s	Run range
Grouped split-K tree attention	3	26.82	26.67–27.11
Fused draft top- k selection	1	29.40	—
Precomputed tap-operand preparation	1	29.02	—

TABLE VI

MEAN EXECUTION SPANS OF EXPLORATORY REPLAY VARIANTS, IN MILLISECONDS. EACH PHASE POOLS ITS OWN DURATION AND TIMER COUNT. DEPLOYED USES TWO INSTRUMENTED RUNS; STOCK ATTENTION THREE; OTHER VARIANTS ONE.

Configuration	Target	Draft	Dispatch	Step
Deployed	112.14	49.84	20.81	189.35
Stock attention	126.78	49.94	21.13	204.44
Fused selection off	112.32	50.57	21.08	190.38
Operand preparation off	111.85	49.85	20.75	188.97

Boot logs report 20.54 GiB of model-loading allocation in every variant. Reported CUDA-graph pools are 1.52–1.59 GiB for the deployed runs, 0.42–0.50 GiB for stock attention, 1.97 GiB without fused selection, and 1.47 GiB without prepared operands. Deployed KV capacity is 203,776 tokens versus 197,632–198,656 for stock attention. These allocation reports do not measure peak serving memory, and the unequal capacities further limit attribution.

Output-side counters pooled over the repeated vLLM runs report 4.47 accepted drafts per speculative event for LumoTree and 3.17 for five-token MTP. These counters describe emitted drafts, excluding the correction or bonus token, and do not identify the committed path. SGLang reports only windowed acceptance averages, so no corresponding cumulative acceptance comparison is reported.

a) Disjoint request corpus: Table VII reports a separately frozen 43-request corpus from four additional Astropy task conversations. Across three runs per vLLM arm, LumoTree records 30.74 pooled tokens/s versus native MTP-5’s 27.52, an 11.7% difference. Every individual tree rate exceeds every MTP-5 rate in these runs. This second corpus supports the direction of the observed replay difference, without establishing statistical significance, equal output distributions, or task-completion improvement.

SGLang records 30.24 tokens/s in its single run. Limited cross-stack replication, different output lengths, and pooled duration weights preclude a superiority or equivalence claim. The small pooled tree–SGLang difference changes sign between the two corpora. SGLang’s serialized prompt lengths exceed vLLM’s by 33–156 tokens here, unlike the exact-token-ID comparison in Table IV.

b) Drafting-pass sensitivity: The deployed tree’s maximum depth is eleven, but neural drafting uses only five MTP passes; suffix lookup supplies the deeper tail. We therefore measure three-pass drafting and a suffix-dependent

TABLE VII

DISJOINT 43-REQUEST REPLAY. RATES POOL MATCHED TOTALS; RANGES ARE INDIVIDUAL RUN RATES. REQUEST MESSAGES ARE SHARED, BUT SGLANG USES A DIFFERENT CHAT SERIALIZATION. THE MEASURED TREE SAMPLER USES APPROXIMATE PENALTY HISTORIES.

Configuration	Runs	Tokens/s	Run range
LumoTree	3	30.74	29.92–31.67
vLLM MTP, 5 tokens	3	27.52	27.41–27.71
SGLang EAGLE, 7/8	1	30.24	—

TABLE VIII

EXPLORATORY DRAFTING-PASS VARIANTS ON THE FIRST 43-REQUEST CORPUS. THE DEPLOYED ROW POOLS TWO RUNS; EACH VARIANT HAS ONE. GPU DRAFTING AND STEP WALL DURATIONS USE THEIR OWN TIMER COUNTS.

Policy	Tokens/s	Accepted drafts/event	Draft ms	Step ms
Five passes	28.69	4.45	49.84	189.35
Three passes	28.94	3.97	32.01	171.27
Conditional three/five	27.05	4.09	47.71	187.55

three/five-pass policy while retaining 32 physical verification rows (Table VIII). These are drafting-policy variants, not measurements of newly constructed 16-node or shallower tree topologies.

Three-pass drafting saves approximately 18 ms per cycle but reduces accepted drafts per event by about 11%. Its 0.9% pooled-rate difference from the deployed runs is small relative to the observed variation of the repeated configuration; no reliable gain is established. The conditional policy has a lower pooled rate. The five-pass configuration is retained. An initial shortened-pass implementation duplicated principal tokens into inactive sibling leaves, producing misleadingly low acceptance; the reported variants use the repaired implementation. Those earlier measurements and the correction remain in the supplement.

C. Acceptance and execution costs

The task study emits an average of 4.12 accepted drafts per LumoTree speculative event. This output-side counter excludes the correction or bonus token and does not identify the branch published into persistent state. Physical draft slots include padding and are distinct from logical candidates.

Table IX characterizes the complete LumoTree cycle on these workloads. Dispatch combines sampling and native replay; the GPU spans do not form an isolated critical-path decomposition. Acceptance and phase durations vary with the request corpus, so higher replay acceptance is not evidence of better task quality.

D. Numerical and candidate-selection validation

We test GDN verification and accepted recurrent-state publication on two calibration and two held-out synthetic operand sets. Each set supplies 48 recurrent instances with the evaluated tree geometry: 32 physical output rows, including

TABLE IX

LUMOTREE ACCEPTANCE AND MEAN EXECUTION SPANS. DISPATCH INCLUDES ACCEPTANCE, PATH HANDLING, AND STATE COMMITMENT. GPU TIMERS DIVIDE EACH DURATION SUM BY ITS OWN SPAN COUNT; WALL TIME COVERS THE DECODING STEP. DISJOINT REPLAY POOLS THREE RUNS.

Metric	Task study	Disjoint replay
Accepted drafts per event	4.12	4.72
Target GPU span (ms)	114.05	109.77
Drafter GPU span (ms)	54.26	48.17
Dispatch GPU span (ms)	20.16	20.94
Step wall time (ms)	195.91	185.05

TABLE X

GDN COMPONENT CHECKS ON GB10. BITWISE COLUMNS COMPARE WITH THE NATIVE CHAIN; THE PAIRED RULE COMPARES BOTH IMPLEMENTATIONS WITH A FLOAT64 REFERENCE. COUNTS ARE PER BLOCK AND PROCESS.

Operand sets	BF16 output bitwise	FP32 state bitwise	Paired rule passed
Calibration	3,004/3,072	2,688/2,688	5,760/5,760
Held-out	3,018/3,072	2,688/2,688	5,760/5,760

padding, and 28 accepted publication paths. Each block runs twice in each of two fresh processes. These repetitions check reproducibility; they do not increase the number of distinct inputs.

For each head, we compare both implementations with a float64 recurrence. We require both root-mean-square and maximum absolute errors to satisfy

$$E_{\text{tree}} \leq 1.10E_{\text{native}} + 2^{-24}M_{64} + 2^{-149}, \quad (6)$$

where M_{64} is the corresponding root-mean-square or maximum magnitude of the reference. Table X reports all enumerated cases passing in both processes. All published recurrent states are bitwise equal to the native chain, while 68 calibration and 54 held-out output tensors differ in BF16 rounding. Native and candidate results reproduce bitwise across repeats and processes.

Calibration controls detect sibling substitution, swapped state rings, and stale publication metadata. Poisoning an off-path branch leaves the tested selected-path state unchanged, and a zero-replay control is rejected. These results support the tested GDN scan and recurrent publication mechanism.

The fused full-vocabulary selection test compares the two output tensors against the reference argmax and top-k operations. Across 1,368 input cases and five block-count settings (6,840 configurations), it records zero byte mismatches; planted negative controls fire. A separate captured four-level selection graph completes 24 replays with zero mismatches. The reference selection outputs match exactly on these inputs and graph replays.

Attention tests use the same kernel build as the ten-task deployment. They show bit-identical repeated output and log-sum-exp values within each of sixteen determinism cases, with matching digests across two processes. On synthetic query/key/value tensors drawn at scales measured from model

TABLE XI

FULL-MODEL CONTINUATION OVER 126 CYCLES. MAXIMUM CANDIDATE ERROR AND CANDIDATE-TO-NATIVE-ENVELOPE RATIO MAY OCCUR AT DIFFERENT CYCLES. EVERY RATIO IS BELOW THE FROZEN LIMIT OF 2. STATE ERRORS ARE RELATIVE L_2 ; KL IS IN NATS.

Surface	Maximum error	Maximum ratio
GDN recurrent state	0.435	1.08
Convolution history	0.304	1.22
KV, newly committed positions	0.375	1.22
KV, earlier positions	0.234	1.17
Next-token KL divergence	0.111	0.74

operands, 93.31% of output elements agree with the reference kernel within two ULP, the maximum absolute output difference is 2^{-8} , and maximum log-sum-exp disagreement is four ULP. All nine checks pass, including comparisons against a float64 dense reference and checks for non-finite disagreements. Split-K changes the reduction order, so its bounded error accompanies the repeatability result.

Runtime traces confirm execution of the specialized kernel in all sixteen attention layers, fused selection, device acceptance, and captured state publication during the workload runs.

E. Full-model continuation

The full-model comparison evaluates six fresh prefixes of approximately 23,000–46,000 tokens. For each prefix, three schedules replay harvested natural accepted paths over seven cycles, including a final root-only cycle that consumes the last pending token. The 18 schedules therefore cover 126 continuation cycles. Two schedules import a common starting state; the third resumes from each engine’s own prefix-cache state. Every candidate forward is checked against all 32 recorded tree-input rows. These are replayed path schedules, not a test of the live sampler’s complete output distribution. Appendix C specifies the five serving processes and controls.

For state or logit surface s at cycle c , the frozen reference envelope is

$$R_s(c) = \max\{F_s, d_s(B, A; c), d_s(V, A; c), d_s(P, A; c)\}, \quad (7)$$

where A is the sequential reference, B a separate process, V a native cache-capacity control, and P a native packed-kernel variant. State distance is the maximum per-layer relative L_2 error, with floor $F_s = 0.02$; next-token KL divergence uses $F_s = 0.15$ nats. The candidate must satisfy $d_s \leq 2R_s$ on every surface and cycle. Constants were calibrated on an earlier collection and frozen before these fresh prefixes were evaluated.

All 126 cycles satisfy the numerical rule (Table XI), and all ten injected faults are detected at their mutation cycles. The candidate greedy token matches A on 122/126 cycles; the four disagreements satisfy the frozen logit-margin allowance. The separate-process, cache-capacity, and packed-kernel controls agree on 119, 124, and 119 cycles, respectively. Thus the result establishes agreement under the stated bounds, not bitwise model equality or strict greedy-output equality.

The ten faults comprise five types on two prefixes: omitted convolution gathering, omitted recurrent replay, incorrect KV remapping, stale root input, and sibling-state publication. Every cache-resumed case starts with at least 91.9% of its prefix cached, across all five arms (30 observations). One candidate repeat is bitwise identical over its seven cycles. This covers prefix restoration on the tested inputs, rather than the full cache-admission and eviction lifecycle.

The separate calibration collection does not count as held-out validation; its original inconclusive verdict is preserved in the supplement. MTP drafter state and batch sizes above one are outside this comparison.

F. Sampling scope and deployed penalty histories

a) Scope of exact sampling: The sampling rule is exact for the intended target distribution only when the supplied probabilities are correct. We restrict this claim to penalty-free sampling: presence and frequency penalties are zero, the repetition-penalty multiplier is one, min- p is zero, and probability filters use matching tie conventions. The tree path ignores min- p , and its top- k implementation retains exactly k tokens, whereas native single-row decoding retains boundary ties. These differences must be absent or aligned even in the penalty-free regime. The numerical continuation result does not prove joint output-distribution equality.

The deployed and SWE-bench runs instead use presence penalty 1.0. Their logits processor constructs a flattened draft history, as for a chain, rather than each node’s ancestry. A target row should include the path through its parent; a leaf continuation row should include the leaf itself. Flattening can penalize tokens from sibling branches and omit the current leaf token. The first token of a step has the correct penalty history; subsequent decisions can differ. We therefore describe these runs as departing from exact autoregressive sampling after the first token of each step, without asserting that every subsequent token changes.

b) Captured-row and sampler audit: A passive capture of 40 tree steps on four requests contains 2,480 processed target and continuation rows, including padded slots. All match the flattened-chain history; only 63 coincide with the correct path history. Reconstructed probabilities from captured top-256 logits estimate node-level total variation above 0.01 on approximately 22% of deeper decisions, with maximum 0.421. These estimates use rounded logits and depend on filter tie handling; they are not joint-sequence total variation.

The deployed target-only walk consumes target and continuation logits without a separate draft-probability tensor. Its CPU floating-point oracle is sampled 200,000 times on each of six captured steps, for 1.2 million walks. Tests compare emitted-token counts conditional on visiting a node with the supplied probability row, using an approximate chi-square G -test and a Bonferroni-adjusted per-step threshold of 10^{-3} . Nodes require at least 2,000 visits; rare expected counts are pooled. No departure is detected in 54 node tests against the supplied rows. Comparison with corrected ancestry probabilities rejects five of six steps. All five planted fault types are detected in at least

TABLE XII
SAMPLER-AUDIT FAULT SENSITIVITY ON SIX CAPTURED STEPS. EACH FAULT TYPE IS DETECTED AT LEAST ONCE; DETECTION DEPENDS ON WHICH NODES THE WALK VISITS.

Injected fault	Steps detected
Temperature applied twice	6/6
Target rows shifted	6/6
Fixed residual draw	5/6
Unconditional acceptance	3/6
Correlated random draws	1/6

TABLE XIII
RECURRENT AGREEMENT ON GB10 UNDER DISTINCT SEQUENTIAL REFERENCES. COUNTS GIVE CELLS OUTSIDE THE PAIRED ERROR BOUNDS AMONG 64,512 OUTPUT CELLS AND 6,912 STATE CELLS PER CONFIGURATION, WITH IDENTICAL COUNTS IN BOTH REPEATS. THE TWO REFERENCE GROUPS ARE NOT A COMMON ACCURACY RANKING.

Configuration	Output cells	State cells
<i>Native GPU recurrence reference</i>		
LumoTree	0	0
<i>Sequential software references</i>		
Weaver author-default	53,745	1,154
Weaver (FP32 preparation)	57,707	388
TreeWY author-default	37,528	6,912

one step (Table XII). Non-rejection supports this finite sampler check; it does not prove universal exactness or validate the deployed penalty histories.

An earlier ten-boot, 8,000-continuation output diagnostic failed its frozen temperature-control sensitivity requirement and remains uninformative. The tree sampler also seeds one process-wide generator with zero, so boots with identical request order reuse random sequences. They cannot be treated as independent sampling replicates. The probability-history audit identifies a specific deployed defect despite that uninformative output test.

G. Recurrent agreement under distinct references

We compare recurrent verification and accepted-state publication using held-out synthetic operands spanning 48 recurrent instances and 48 value heads. Verification covers the tree’s 28 active nodes. Three accepted-path publications produce a history of 1, 6, and 11 updates, including Weaver’s accepted replay and TreeWY’s final deferred commit. Each configuration is evaluated twice in one process, with a fresh initial state for each cycle. A numerical cell fixes the recurrent instance, output node or publication, and value head; errors are reduced over the remaining vector or matrix entries.

Each cell must satisfy both root-mean-square and maximum absolute error bounds,

$$E_{\text{method}} \leq 1.10E_{\text{seq}} + 2^{-24}M_{64} + 2^{-149}, \quad (8)$$

where errors are measured against the specified float64 recurrence and M_{64} is its corresponding root-mean-square or maximum magnitude. For LumoTree, the sequential comparator is the native GPU recurrence. Weaver and TreeWY use

TABLE XIV

GDN COMPONENT LATENCY IN MILLISECONDS ON GB10. VERIFY, PUBLISH, AND TOTAL COLUMNS GIVE GRAPH-MODE MEDIANS; EAGER GIVES THE TOTAL MEDIAN WITHOUT VERIFIER GRAPH REPLAY. TREEWY FUSES PUBLICATION OF THE PREVIOUS FRONTIER WITH THE CURRENT VERIFICATION; ITS FINAL FLUSH IS REPORTED SEPARATELY. THESE ARE TIMING OBSERVATIONS UNDER DISTINCT NUMERICAL POLICIES.

Method	Verify	Publish	Total	Eager
LumoTree	12.90	4.49	17.69	17.58
Native path baseline	29.71	2.70	32.41	33.73
PyTorch node baseline	52.32	2.70	55.02	81.04
TreeWY author-default	fused		11.01	11.71
Weaver author-default	2.07	1.38	3.45	5.88
Weaver (FP32 preparation)	4.05	1.40	5.45	9.13

sequential software comparators that do not emulate every tensor-core rounding operation.

TreeWY retains the author implementation’s normalization floor of 10^{-12} and BF16 dot products. Weaver uses TF32 verification, FP32 elementwise accepted replay, and an additive normalization constant of 10^{-6} . We report both its author-default preparation and a variant with FP32 preparation. The two Weaver variants retain their respective preprocessing conventions.

Table XIII reports the cells outside each implementation’s paired bounds. All configurations produce finite tensors and reproduce bitwise between repeats. TreeWY’s BF16 compact commit and its sequential comparator’s FP32 outer-product updates, for example, are different floating-point computations. The normalization and arithmetic conventions are part of the numerical comparison and must be interpreted alongside its reference choice (Section IX).

H. Recurrent verification and publication costs

We measure verification and accepted-state publication on shared synthetic operands for 48 GDN instances with the model’s head dimensions, using the 28 active nodes of the deployed 32-row tree. Each measurement has 20 warmup iterations and 100 timed iterations. CUDA events bracket verification and publication; a 256-MiB buffer is written outside each timed iteration. Graph mode captures the verifier while retaining each method’s publication wrapper; eager mode invokes the verifier directly. Accepted paths cycle through all 28 possible frontiers. Durable state is reset before each measurement and evolves across iterations. These measurements exclude attention, convolution, sampling, model projections, and interleaved execution of the remaining model layers.

The native path baseline uses native recurrent updates with the same path cover but stores every node’s full state. The PyTorch baseline instead updates nodes sequentially in topological order. Both publish by copying the selected leaf into durable state. LumoTree records a 17.69-ms graph-mode total, with a 10th–90th percentile interval of 17.18–17.84 ms, versus 32.41 ms (32.36–32.67) for native paths. Total medians are measured directly; they need not equal sums of region medians.

TABLE XV

NAMED PERSISTENT BUFFER ALLOCATIONS AND NUMERICAL ERROR FOR THE GDN MEASUREMENTS. OUTPUT AND STATE COLUMNS ARE MAXIMUM ABSOLUTE ERRORS AGAINST THE FLOAT64 RECURRENCE; STATE USES THE DEEPEST ACCEPTED PATH. BUFFER TOTALS EXCLUDE OUTPUTS, SEPARATE PAGE POOLS, GRAPH POOLS, AND TEMPORARY PER-CALL ALLOCATIONS; THE NAIVE STATE BANKS INCLUDE RESERVED NULL AND DURABLE SLOTS. THESE ARE NOT PEAK SYSTEM MEMORY OR MEASURED MEMORY TRAFFIC.

Method	Buffers (MiB)	Output error	State error
LumoTree	132.5	4.82×10^{-4}	2.62×10^{-6}
Native path baseline	5,040.0	4.82×10^{-4}	2.62×10^{-6}
PyTorch node baseline	5,040.0	4.82×10^{-4}	6.51×10^{-7}
TreeWY author-default	192.8	6.31×10^{-4}	5.10×10^{-3}
Weaver author-default	25.1	5.28×10^{-4}	5.06×10^{-3}
Weaver (FP32 preparation)	31.1	5.18×10^{-4}	8.55×10^{-7}

TreeWY’s steady step takes 11.01 ms. A finite sequence also requires its initial verification without a pending commit and a final deferred publication; the latter takes 10.51 ms in this implementation. Weaver includes accepted-path replay in its total. Its graph-mode topology rebuild takes approximately 0.02 ms separately and is excluded from the table because the topology is reused. These boundaries must be retained in a complete-cycle comparison.

Table XV sums the tensor allocations explicitly retained for verification and publication. LumoTree’s 96-MiB cut-state buffer is reused across layers; only five cut rows, totaling 15 MiB per layer, are logically exported. This agrees with the storage formula in Section IV, but does not measure physical memory transactions. Operand rings and publication staging contribute the remaining named buffers. The deployment’s separately allocated 1.5-GiB recurrent page pool is excluded. The naive baselines retain a 34-slot bank (32 physical nodes, one null slot, and one durable slot) and a staging copy for publication. Thus allocation boundaries differ across implementations. Framework peak allocations and all raw timing samples are preserved in the supplement.

The errors in this experiment are descriptive, rather than an admission criterion. TreeWY and the two Weaver variants retain the preprocessing and precision differences described in Section VII-G. Consequently, faster timings here do not establish equal-accuracy superiority, and they do not overturn the paired-bound failures in Table XIII. These isolated GDN measurements also do not predict a full-model decoding speedup.

VIII. DISCUSSION

The design couples tree geometry, physical layout, and continuation publication. Optimizing any one of these in isolation can move work elsewhere or invalidate another stage’s mapping. Transient cut states, graph buffers, and native persistent rows serve different purposes: their separation makes the storage and recomputation tradeoff explicit. Path scans retain sequential arithmetic, while native replay pays accepted-

path update work to return persistent state to the serving representation.

For coding agents, decoding rate and task completion measure different effects. Additional tokens can reflect more reasoning or a longer tool loop, and tool execution contributes to agent duration without entering the decode denominator. Recorded-request replay fixes incoming messages but does not execute the newly generated tools or subsequent agent decisions. The measured 10.2% and 11.7% rate advantages over five-token native MTP therefore apply to the two request corpora and serving configuration. The penalty-history discrepancy means these are comparisons of the implemented sampling procedures, rather than equal target-distribution comparisons. They do not transfer to a task-level speed claim: in the live agent study, LumoTree generates more tokens, takes longer in aggregate, and resolves five tasks compared with MTP’s six. Task outcomes and GPU spans describe complementary parts of the serving/application boundary.

IX. LIMITATIONS

a) Numerical scope: The component results cover recurrent verification and publication, attention error, and candidate selection. The held-out full-model comparison covers 126 replayed continuation cycles with sensitive fault controls. It permits native-relative numerical error and margin-bounded greedy disagreements, rather than establishing bitwise or strict greedy equality. Harvested paths do not validate the live acceptance loop or MTP drafter state, and 18 schedules do not exhaust every path combination. Prefix restoration is covered on the tested prompts; cache admission and eviction, row reuse, batch sizes above one, and complete distribution preservation remain outside that result. The deployed penalty-history defect makes the measured tree sampling approximate after the first token of each step. Exactness is restricted to the penalty-free, filter-matched regime and remains conditional on correct supplied probabilities. Finite sampler non-rejection cannot prove universal exactness. The paired recurrent tests use different sequential comparators and preprocessing conventions, so they do not establish a common accuracy ranking. Their isolated timings omit the rest of the hybrid model. Matching starting states, tokens, and positions does not guarantee identical intermediate hidden tensors or identify the cause of native variation.

b) Performance and task scope: The task study contains one sampled attempt per task and arm, ten selected Astropy tasks, and no plain-decoding agent control. It provides neither a benchmark-wide quality estimate nor an isolated task-completion speedup: the arms execute different request and tool trajectories. Both replay corpora have three repetitions per principal vLLM arm; the disjoint corpus has one SGLang run and is drawn from four additional task conversations within the same project. Repeated tree boots reuse a process-seeded random sequence and are not independent sampling replicates. This limited replication does not establish significance, SGLang parity, or broad workload generalization. Sampling changes output lengths, and the response cap truncates some requests.

Stream buffering and parsing enter the client-side duration; subtracting one token approximates the first output interval when a speculative chunk contains several tokens. Exact input IDs remove the serialization discrepancy only in the first corpus’s SGLang comparison. The disjoint replay and live agent study use separate chat templates, attention and prefill implementations. Optimization variants have limited repetitions and do not isolate every transformation. Equal memory-fraction settings do not establish equal KV-block capacity. GPU instrumentation overhead has not been measured. Boot allocation logs do not establish peak serving memory. Named scratch allocations exclude other resident buffers and temporary work; logical state-export bytes are not measurements of memory traffic.

X. CONCLUSION

LumoTree combines path-parallel recurrent scans with native replay and a shared mapping for convolution, attention, acceptance, and drafter continuation. Sampled replay on GB10 yields 28.79 versus 26.12 pooled tokens/s for five-token native MTP, and 30.74 versus 27.52 on a disjoint corpus, with three runs per arm. The 30-attempt task study records 26.19 versus 27.17 tokens/s and five versus six resolved tasks. Full-model continuation satisfies frozen native-relative numerical bounds over 126 cycles and detects all ten injected faults. Sampling exactness is restricted to penalty-free settings with matching filters and correct target probabilities; the measured presence-penalty-1.0 runs depart from exact sampling after the first token of each tree step. The results characterize bounded numerical agreement and replay rates of the implemented system, without claiming lossless deployed sampling or task-completion speedup.

APPENDIX A

SERVING CONFIGURATION AND MEASUREMENT

The implementation artifact [37] documents the serving route and component studies. This revision’s source and data supplement preserves the additional task study, recorded-request replays, continuation diagnostics, and recurrent cost measurements. Input manifests, run selections, pooled-rate reductions, allocation accounting, raw timing samples, and failed or invalid observations remain separate from the manuscript’s scientific claims.

All measured tree runs use presence penalty 1.0. Exact-sampling claims apply only to penalty-free, filter-matched settings with correct target probabilities; the measured tree deployment departs from exact sampling after the first token of each step (Section VII-F).

Workload rates use Eq. 5; token counts and latency sums cover the same completed requests. Metric counters are checked for continuity, resets, and outstanding requests at task boundaries. First-output latency, tools, and evaluator time remain distinct from decode duration. GPU phase times divide each timer’s duration sum by its own span count. Output-side acceptance counters describe emitted draft lengths and do not identify the committed tree path.

TABLE XVI

SERVING SETTINGS FOR THE THREE-ARM TASK STUDY. NATIVE MTP AND LUMOTREE SHARE THE VLLM BUILD. EXACT VERSIONS AND LAUNCH SETTINGS ACCOMPANY THE SUPPLEMENT.

Setting	Configuration
Target	Qwen3.8-27B; mixed NVFP4/FP8 weights, BF16 execution
Drafter	Same-checkpoint one-layer BF16 MTP in all arms; no separate EAGLE draft weights
LumoTree proposals	27 drafts plus root, padded to 32 rows; maximum depth 11; five MTP passes and suffix extension
Native proposals	MTP: five drafts; SGLang: seven steps, top- k one, capacity eight drafts
Attention	vLLM: forked FlashAttention-2; LumoTree adds grouping 2 and split-K 4; SGLang: FlashInfer
Scheduling	One active sequence; 4,096 prefill tokens for vLLM, 8,192 for SGLang
Context / memory	131,072 tokens; memory fraction 0.70; actual KV capacities not equalized
Agent / budget	Qwen Code 0.19.4; one task at a time; 9,000 seconds
Response cap	24,000 tokens per request
Sampling	Temperature 0.6; top- p 0.95; top- k 20; min- p 0; presence penalty 1.0

For target probabilities p and proposal probabilities q , ordinary rejection sampling combines accepted mass $\min(p, q)$ with residual mass proportional to $(p - q)_+$. This gives the target marginal for the probabilities actually supplied [1], [2]. Serving additionally requires that subsequent forwards consume the state associated with the accepted path and that logits processing constructs that path’s target distribution. Our exact-sampling claim is limited to the penalty-free, filter-matched regime; the history-dependent measured deployment is approximate as described in Section VII-F.

APPENDIX B

ADDITIONAL WORKLOAD RESULTS

Table XVII retains every attempt in the three-arm task study. Agent duration includes model service and tool execution, excluding server startup and subsequent evaluation. Empty-patch outcomes are distinguished from patches that reach the test evaluator.

TABLE XVII

TASK-LEVEL OUTCOMES. IDS HAVE PREFIX ASTROPY__ASTROPY-. EACH CELL GIVES ROUNDED AGENT MINUTES AND OUTCOME: R, RESOLVED; F, FAILED TESTS; E, EMPTY PATCH.

Task suffix	LumoTree	MTP-5	SGLang
13977	8 / F	10 / F	21 / F
14096	24 / R	20 / E	10 / E
14182	6 / F	6 / E	1 / E
14309	3 / R	2 / R	2 / R
14365	3 / F	2 / R	3 / R
14369	32 / F	18 / R	16 / F
14508	37 / R	7 / R	7 / R
14539	6 / R	3 / R	17 / R
14598	21 / F	27 / F	20 / F
14995	4 / R	5 / R	5 / R

APPENDIX C

FULL-MODEL COMPARISON PROTOCOL

The continuation comparison uses five fresh serving processes: sequential reference A , an independent process B , a native cache-capacity control V , a native packed-GDN-kernel variant P , and LumoTree. The reference uses a 40 GiB KV allocation; V uses the candidate’s observed allocation. Candidate capacity follows its deployed memory-fraction policy. All arms consume matching forced tokens and positions and retain their own published state across subsequent cycles.

Six prefixes from four additional Astropy task conversations are disjoint from the earlier continuation and first replay corpora. A separate harvest records natural tree inputs and accepted paths. Each prefix supplies three seven-cycle schedules: a cold salted prefix with a common starting-state import, an unsalted repeat that primes the cache with the same import, and a cache-resumed case with no import. The last cycle accepts only the root to consume the final pending token; next-forward logits also observe the subsequent pending token. The root-only cycle is included in the 126-cycle total. Candidate-input checks compare every one of the 32 verification rows with the recorded tree.

The frozen numerical rule uses the largest same-cycle native-control distance, floored at 0.02 for state relative L_2 and 0.15 nats for next-token KL. Candidate distance must not exceed twice this envelope. A greedy mismatch is permitted only when A ’s top-two logit margin is within $\max(0.25, 2 \max_{B,V,P} \|\Delta \text{logit}\|_\infty)$ or the native controls are categorically unstable outside that band; at most 10% of cells may be native-unstable. Input parity, complete sealed observations, starting-state read-back, and detection of every injected fault are prerequisites. These allowances make the verdict a numerical agreement test, rather than exact token or logit equality.

Five fault types run on two prefixes each at a selected natural cycle: omitted convolution gathering, omitted recurrent replay, incorrect KV remapping, stale root input, and sibling-state publication. Each must exceed the same base case’s bound on its designated surface; sibling publication may exceed any of its three targeted state surfaces. Faults are reported separately from the 126 clean cycles. Prefix-cache coverage requires at least 90% cached tokens in every cache-resumed observation. One clean candidate case is repeated in the same process. Earlier invalid forcing and insensitive comparison records remain in the supplement with their original dispositions.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 19 274–19 286. [Online]. Available: <https://proceedings.mlr.press/v202/leviathan23a.html>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>

- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. ACM, 2024, pp. 932–949. [Online]. Available: <https://doi.org/10.1145/3620666.3651335>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, "Gated delta networks: Improving Mamba2 with delta rule," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=r8H7xhYPwz>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, "STree: Speculative tree decoding for hybrid state-space models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Wang, X. Han, Z. Yang, F. Liu, X. Li, R. Gu, S. Zhong, and C. Tian, "SpecLA: Efficient Speculative Decoding for Linear-Attention Models," 2026. [Online]. Available: <https://arxiv.org/abs/2607.16673v1>
- [7] Y. Oda, R. Mathieu, R. Knyazhitskiy, and A. Chakhvadze, "Trees from Marginals: Autoregressive drafting with factorized priors," 2026, technical report. [Online]. Available: <https://arxiv.org/abs/2607.06763>
- [8] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQm66>
- [9] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, "Sequoia: Scalable and robust speculative decoding," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 129 531–129 563. [Online]. Available: <https://doi.org/10.52202/079017-4116>
- [10] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster inference of language models with dynamic draft trees," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 7421–7432. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.422/>
- [11] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, "OPT-Tree: Speculative decoding with adaptive draft tree structure," 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [12] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-3: Scaling up inference acceleration of large language models via training-time test," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 136 737–136 756.
- [13] J. Chen, Y. Liang, and Z. Liu, "DFlash: Block Diffusion for Flash Speculative Decoding," 2026. [Online]. Available: <https://arxiv.org/abs/2602.06036>
- [14] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [15] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 62 557–62 583. [Online]. Available: <https://doi.org/10.52202/079017-2000>
- [16] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, "Marconi: Prefix Caching for the Era of Hybrid LLMs," 2025. [Online]. Available: <https://arxiv.org/abs/2411.19379>
- [17] SGLang Team, "Hybrid Models Meet SGLang: More than Full Attention," <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [18] SGLang Project, "Qwen3-Next SGLang documentation," <https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [19] T. Dao, D. Haziza, F. Massa, and G. Sizov, "Flash-Decoding for long-context inference," <https://pytorch.org/blog/flash-decoding/>, 2023, October 13.
- [20] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and customizable attention engine for LLM inference serving," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025.
- [21] J. Yao, K. Chen, K. Zhang, J. You, B. Yuan, Z. Wang, and T. Lin, "DeFT: Decoding with flash tree-attention for efficient tree-structured LLM inference," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=2c7pfOqu9k>
- [22] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Z. Yu, X. Tang, Y. Wang, and Y. Ding, "FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference," in *Proceedings of Machine Learning and Systems*, vol. 7, 2025.
- [23] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=mZn2Xyh9Ec>
- [24] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, "Parallelizing linear transformers with the delta rule over sequence length," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 115 491–115 522. [Online]. Available: <https://doi.org/10.52202/079017-3668>
- [25] Y. Wu, Y. Dukler, M. Trager, A. Achille, W. Xia, and S. Soatto, "Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding," in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, vol. 262. PMLR, 2024, pp. 292–304. [Online]. Available: <https://proceedings.mlr.press/v262/wu24a.html>
- [26] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, "SpecMamba: Accelerating Mamba inference on FPGA with speculative decoding," 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [27] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, "Component-aware self-speculative decoding in hybrid language models," 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [28] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, "Bole: Efficient Tree Speculation for Hybrid-Attention Language Models," 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [29] S. M. Ghantasala, "TreeWY: Speculative Verification for Gated DeltaNet Hybrids," 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [30] —, "TreeWY: Tree Speculative Decoding for Hybrid GDN Models," <https://github.com/vllm-project/vllm/issues/54080>, 2026, author implementation RFC; accessed September 26, 2026.
- [31] Dao AI Lab and NVIDIA, "ReplaySSM: Cache SSM Inputs Instead of State," <https://dao-lab.ai/blog/2026/replayssm/>, 2026, author technical report; accessed September 26, 2026.
- [32] X. Yang, C. Peng, Y. Zhao, L. Zeng, A. Hu, J. Yang, S. Wang, J. Lv, X. Liang, C. Yang, J. Liu, and Y. Qiu, "OneLA: Scaling Linear-Attention Decoding to Large Beams in Generative Recommendation," 2026. [Online]. Available: <https://arxiv.org/abs/2609.12399v1>
- [33] H. He, "Defeating Nondeterminism in LLM Inference," <https://thinkngmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025, Thinking Machines Lab, September 10, 2025.
- [34] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, "SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications," 2025. [Online]. Available: <https://arxiv.org/abs/2411.04975v3>
- [35] Snowflake Arctic Inference, "Suffix Decoding," <https://arcticinference.readthedocs.io/en/latest/suffix-decoding.html>, 2026, implementation documentation; accessed September 24, 2026.
- [36] Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen, and J. Zhang, "SAM Decoding: Speculative Decoding via Suffix Automaton," 2024. [Online]. Available: <https://arxiv.org/abs/2411.10666v3>
- [37] Z. Ma, "LumoTree: Implementation and Experiment Artifact," https://github.com/MaCoredroid/Lumo_FlyWheel/tree/lumotree-v2-artifact-r1/papers/gdn-tree-scan-mlsys/v2, 2026, version 1; paper source, evaluated implementations, and supporting experiment records.